AI Lab assignment 5

Tejas Patil
U24AI061

Q1)

```cpp
#include <bits/stdc++.h>
using namespace std;

struct State {
    int gLeft, bLeft;
    bool boatLeft;

    bool operator<(const State &other) const {
        return tie(gLeft, bLeft, boatLeft) <
               tie(other.gLeft, other.bLeft, other.boatLeft);
    }
};

int exploredStates = 0;

bool isValid(State s) {
    int gRight = 3 - s.gLeft;
    int bRight = 3 - s.bLeft;

    if (s.gLeft < 0 || s.bLeft < 0 || s.gLeft > 3 || s.bLeft > 3)
        return false;

    if (s.gLeft > 0 && s.bLeft > s.gLeft)
        return false;

    if (gRight > 0 && bRight > gRight)
        return false;

    return true;
}

bool isGoal(State s) {
    return (s.gLeft == 0 && s.bLeft == 0 && s.boatLeft == false);
```

```
}

vector<State> getNextStates(State s) {

    vector<State> next;
    vector<pair<int,int>> moves = {
        {2,0}, {0,2}, {1,1}, {1,0}, {0,1}
    };

    for (auto move : moves) {
        State newState = s;

        if (s.boatLeft) {
            newState.gLeft -= move.first;
            newState.bLeft -= move.second;
        } else {
            newState.gLeft += move.first;
            newState.bLeft += move.second;
        }

        newState.boatLeft = !s.boatLeft;

        if (isValid(newState))
            next.push_back(newState);
    }

    return next;
}

bool DLS(State current, int depth, int limit,
         set<State> &visited) {

    exploredStates++;

    if (isGoal(current))
        return true;

    if (depth == limit)
        return false;
```

```
    visited.insert(current);

    for (State next : getNextStates(current)) {
        if (visited.find(next) == visited.end()) {
            if (DLS(next, depth+1, limit, visited))
                return true;
        }
    }

    return false;
}

int main() {

    State start = {3,3,true};
    int limit = 3;

    set<State> visited;

    bool found = DLS(start, 0, limit, visited);

    cout << "Depth Limited Search (limit = 3)\n";

    if (found)
        cout << "Solution Found\n";
    else
        cout << "Solution NOT Found\n";

    cout << "States explored: " << exploredStates << endl;

    return 0;
}
```

```
Depth Limited Search (limit = 3)
Solution NOT Found
States explored: 7
```

Q2)

```cpp
#include <bits/stdc++.h>
using namespace std;

struct State {
    int gLeft, bLeft;
    bool boatLeft;

    bool operator<(const State &other) const {
        return tie(gLeft, bLeft, boatLeft) <
               tie(other.gLeft, other.bLeft, other.boatLeft);
    }
};

map<State, State> parent;
int exploredStates;

bool isValid(State s) {
    int gRight = 3 - s.gLeft;
    int bRight = 3 - s.bLeft;

    if (s.gLeft < 0 || s.bLeft < 0 || s.gLeft > 3 || s.bLeft > 3)
        return false;

    if (s.gLeft > 0 && s.bLeft > s.gLeft)
        return false;

    if (gRight > 0 && bRight > gRight)
        return false;

    return true;
}

bool isGoal(State s) {
    return (s.gLeft == 0 && s.bLeft == 0 && s.boatLeft == false);
}

vector<State> getNextStates(State s) {
```

```
    vector<State> next;
    vector<pair<int,int>> moves = {
        {2,0}, {0,2}, {1,1}, {1,0}, {0,1}
    };

    for (auto move : moves) {

        State newState = s;

        if (s.boatLeft) {
            newState.gLeft -= move.first;
            newState.bLeft -= move.second;
        } else {
            newState.gLeft += move.first;
            newState.bLeft += move.second;
        }

        newState.boatLeft = !s.boatLeft;

        if (isValid(newState))
            next.push_back(newState);
    }

    return next;
}

bool DLS(State current, int depth, int limit,
         set<State> &visited) {

    exploredStates++;

    if (isGoal(current))
        return true;

    if (depth == limit)
        return false;

    visited.insert(current);
```

```
    for (State next : getNextStates(current)) {

        if (visited.find(next) == visited.end()) {

            parent[next] = current;

            if (DLS(next, depth+1, limit, visited))
                return true;
        }
    }

    return false;
}

void printPath(State goal) {

    vector<State> path;
    State cur = goal;

    while (!(cur.gLeft == 3 && cur.bLeft == 3 && cur.boatLeft == true)) {
        path.push_back(cur);
        cur = parent[cur];
    }

    path.push_back(cur);
    reverse(path.begin(), path.end());

    cout << "\nSolution Path:\n";
    for (int i = 0; i < path.size(); i++) {
        cout << "("
             << path[i].gLeft << "G, "
             << path[i].bLeft << "B, Boat "
             << (path[i].boatLeft ? "Left" : "Right")
             << ")\n";
    }
}

int main() {

    State start = {3,3,true};
```

```
    for (int depth = 0; depth <= 20; depth++) {

        set<State> visited;
        parent.clear();
        exploredStates = 0;

        if (DLS(start, 0, depth, visited)) {

            cout << "Solution found at depth: "
                 << depth << endl;

            cout << "States explored: "
                 << exploredStates << endl;

            State goal = {0,0,false};
            printPath(goal);
            return 0;
        }
    }

    return 0;
}
```

```
Solution found at depth: 11
States explored: 12

Solution Path:
(3G, 3B, Boat Left)
(3G, 1B, Boat Right)
(3G, 2B, Boat Left)
(3G, 0B, Boat Right)
(3G, 1B, Boat Left)
(1G, 1B, Boat Right)
(2G, 2B, Boat Left)
(0G, 2B, Boat Right)
(0G, 3B, Boat Left)
(0G, 1B, Boat Right)
(1G, 1B, Boat Left)
(0G, 0B, Boat Right)
```